

DEEP LEARNING

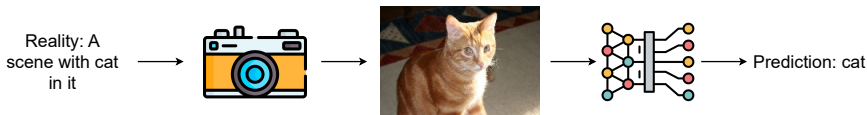
Deep Learning for Computer Vision: Part I

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

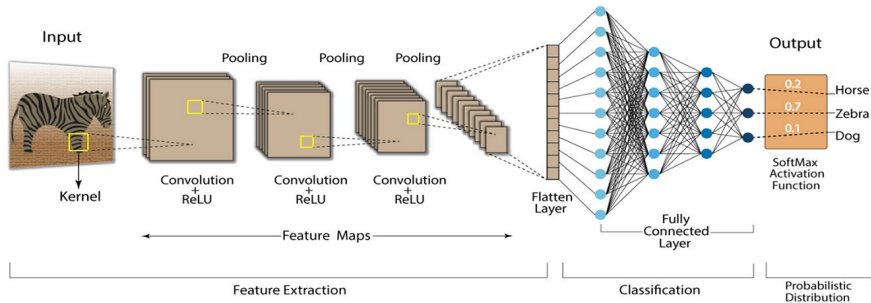
Recap: What is Computer Vision?

- **Goal:** Enable machines to see, understand, and act on visual data using neural networks.
- Core idea in computer vision:
 - ▶ Image in $\rightarrow$ prediction out
 - ▶ Learn features automatically using a deep neural network
 - ▶ Hierarchical representation: We go from pixels $\rightarrow$ edges $\rightarrow$ textures $\rightarrow$ object parts $\rightarrow$ objects



- Applications:
 - ▶ Autonomous driving
 - ▶ Medical imaging
 - ▶ Surveillance

Recap: Simple Convolutional Neural Network

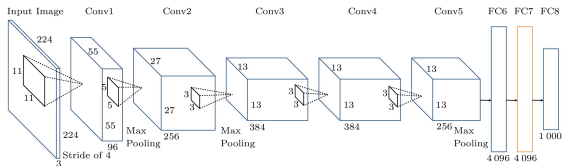


CNNs can be divided into the following parts:

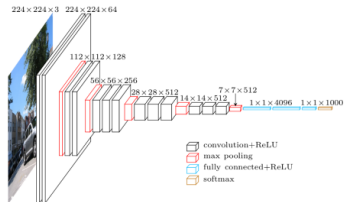
- **Feature Extractor:** Repeating patterns of Convolution → ReLU → Pooling blocks.
- **Classifier:** A flattening operation, followed by fully connected layers. The output layer contains as many neurons as the number of classes we want to predict.
- **Output Probability Distribution:** A softmax function is applied to the output logits to get a probability distribution over the classes.

Recap: Advanced CNN

- CNNs becomes more powerful if we add more convolution layers.
- Popular CNNs for the task of image classification: Alexnet, VGG, ResNet.



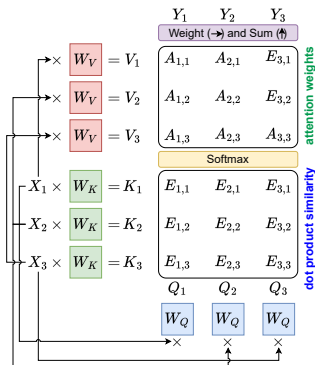
Alexnet (60 M parameters)



VGG16 (138 M parameters)

Recap: Self-Attention

- The self-attention mechanism allows every input token to **attend** to every other token in the input sequence. This allows the model to understand the relationships between tokens and context.
- The attention weights are computed using the formula: $A = \text{softmax}(\frac{QK^T}{\sqrt{d}})$

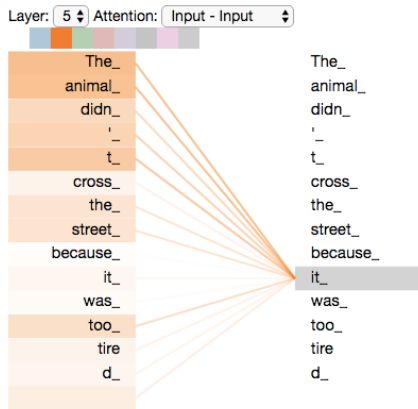


	He	swung	the	bat	with	incredible	force
He	0.35	0.10	0.05	0.05	0.05	0.05	0.35
swung	0.10	0.25	0.10	0.25	0.10	0.10	0.15
the	0.05	0.10	0.40	0.25	0.10	0.05	0.05
bat	0.05	0.20	0.10	0.30	0.15	0.10	0.10
with	0.05	0.10	0.10	0.15	0.30	0.20	0.10
incredible	0.05	0.10	0.05	0.10	0.25	0.35	0.10
force	0.35	0.05	0.05	0.05	0.05	0.10	0.35

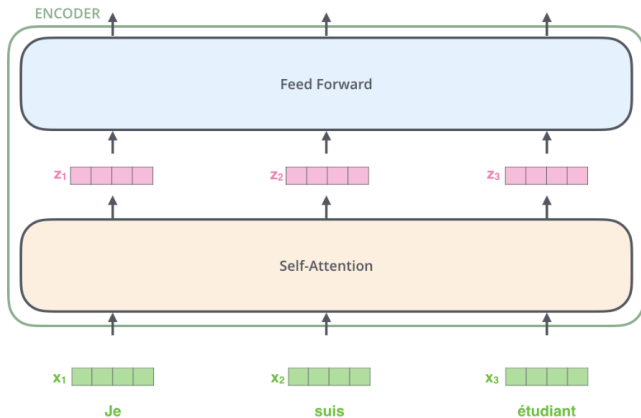
sum along rows = 1

Recap: Visualizing Self-Attention

- We can visualize the attention learned by the model by training on a large enough dataset.
- Consider an example sequence:
"The animal didn't cross the street because it was too tired"
- We can observe that when the model is processing the word (or token) "it", self-attention allows it to associate "it" with "animal" (darker color represents higher importance).



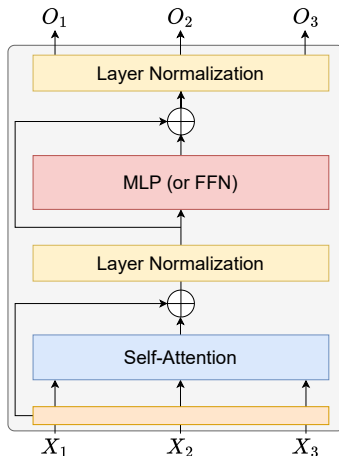
Recap: Transformer Encoder Block



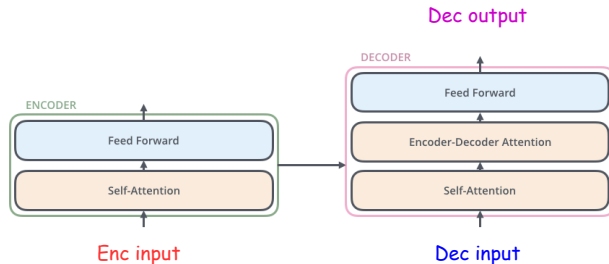
- Each encoder block takes an input a matrix (input size: seq. length $\times$ embedding size), and outputs a matrix of the same size.

Recap: Transformer Encoder Block

- The transformer block is repeated several times in the whole architecture.
- It consists of the following components:
 - ▶ Attention (has several variants: self-attention, cross-attention, masked self-attention and multi-head attention)
 - ▶ Layer Normalization
 - ▶ MLP (or FFN)
 - ▶ Residual Connections
 - ▶ Positional Encoding (plus the input embeddings that convert word to numbers)

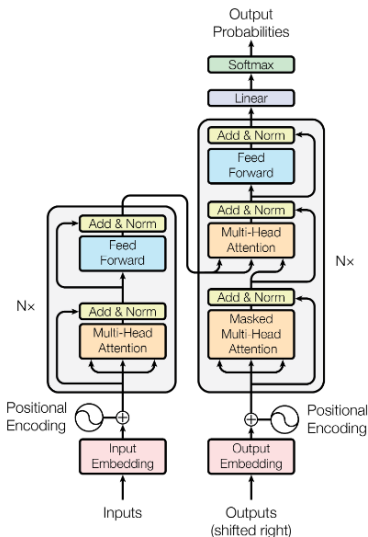


Recap: Encoder-Decoder Transformer

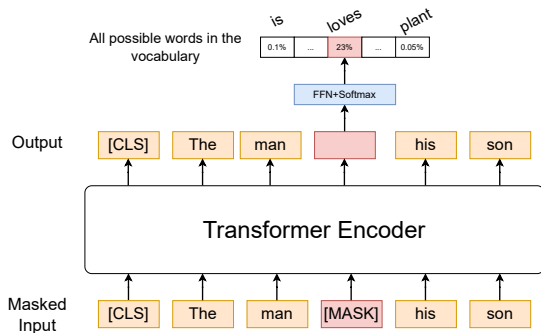


Enc input	Dec input	Dec output
"The chicken crossed the street"	[START]	Il
"The chicken crossed the street"	[START] Il	pollo
"The chicken crossed the street"	[START] Il pollo	attraversa
...	...	...

Recap: Encoder-Decoder Transformer



Recap: Encoder-only Transformer



- BERT uses a **encoder-only** transformer architecture that is trained using **masked language modelling** (MLM), where we randomly mask 15% of the input tokens. We ask the model to predict the masked tokens.
- Note, we introduce a special token (or embedding) called class token: [CLS]. This special token summarizes information from the whole input through the self-attention mechanism.

Today

- CNNs are versatile and can be used for tasks beyond image classification, such as:
 - ▶ Object detection
 - ▶ Segmentation (today)
 - ▶ Depth estimation
 - ▶ ... many more
- Transformers although originally developed for the task of NLP, can be used for Computer Vision tasks:
 - ▶ Image classification (today)
 - ▶ Segmentation
 - ▶ Object detection
 - ▶ ... many more



Segmentation



Object detection



Depth estimation

Table of Contents

1. Vision Transformer

2. Segmentation

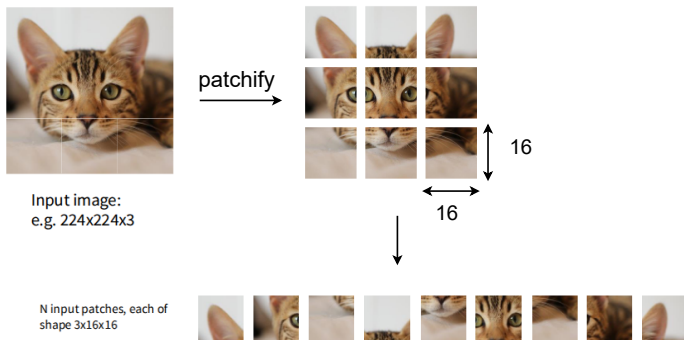
Transformer for Image Classification

Vision Transformer

- Transformer architecture is a very versatile one, and can be used for image classification as well.
- **Question:** How do we convert a RGB image into a sequence?
- **Answer #1:** Treat every pixel as a 3-dimensional vector (1 red channel, 1 blue channel and 1 green channel). And all the pixels together become a sequence.
- **Issue:** Images have a lot of pixels. With this approach the sequence length will become too long.
- **Answer #2:** Flatten the whole image into a single vector.
- **Issue:** The downside is that the resulting flattened vector will be too long, and we do not have a sequence anymore.
- **Solution:** We meet in the middle!

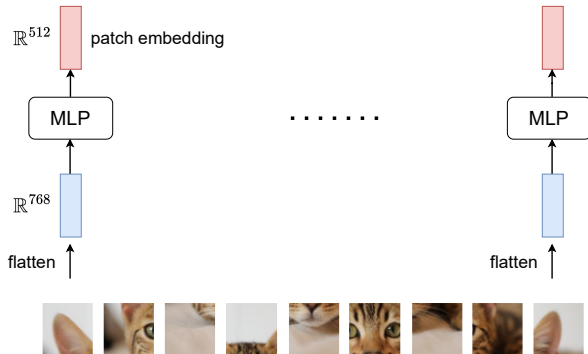
Vision Transformer: Patchification

- In order to prepare an image for the transformer architecture, we **patchify** the input.
- It splits the input into small patches (eg, $16 \times 16 \times 3$).
- For an image of size 224×224 , we get $(224/16)^2 = 196$ patches. Therefore, the sequence length becomes 196.



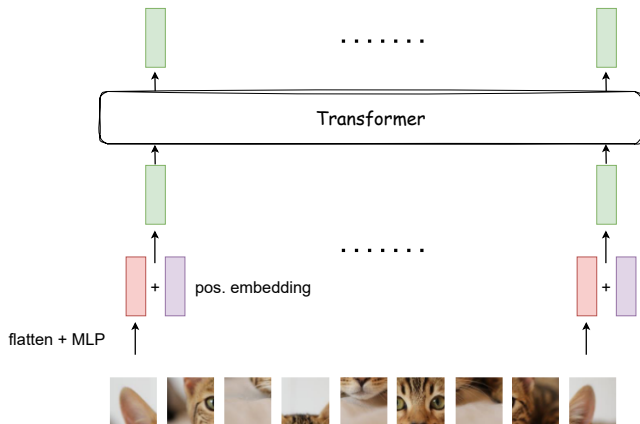
Vision Transformer: Patch Embedding

- We flatten the raw pixels to get a flattened vector (eg, $\mathbb{R}^{768}$).
- We apply a fully connected neural network (or MLP) on the flattened vector to get a **patch embedding** (eg, $\mathbb{R}^{512}$), which goes into the transformer.



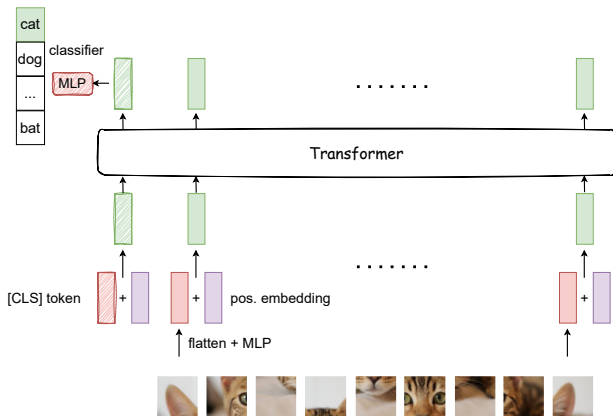
Vision Transformer: Positional Embedding

- To the patch embedding, we add **positional embedding**, which encodes the location of the patch. The positional embedding is learnable.



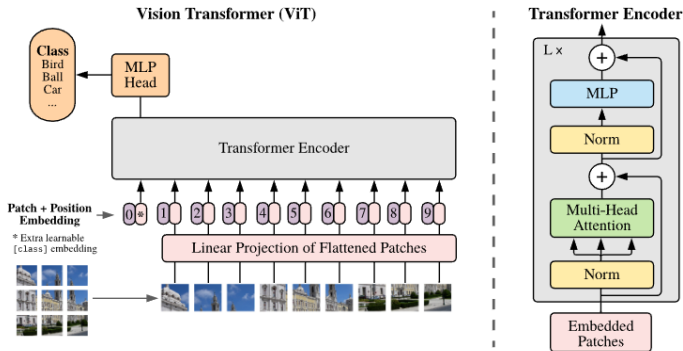
Vision Transformer: [CLS] Embedding

- We add a learnable embedding called the [CLS] token or the [CLS] embedding as an additional input to the transformer.
- We attach a simple MLP classifier to the [CLS] token at the output of the transformer, which acts as a classifier.



Vision Transformer (ViT): An Image is Worth 16×16 Words

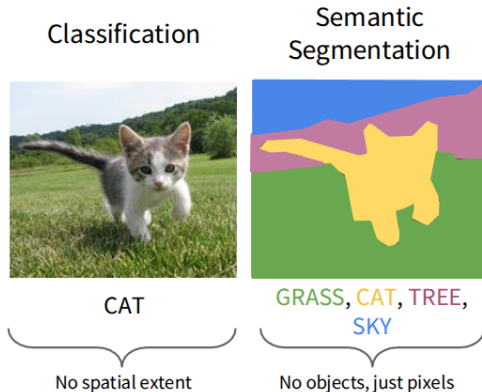
- The paper "An Image is Worth 16×16 Words"¹ first showed how Transformers (encoder-only) from NLP can also be used for image classification.
- The only modification proposed in this work is how to convert images to patches



¹Paper

Segmentation

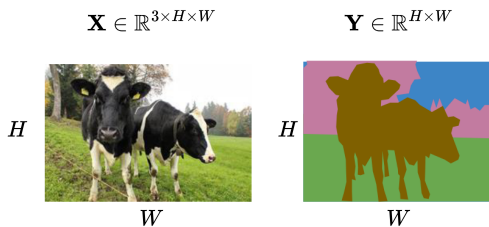
Segmentation: Going Beyond Image-level Prediction



- Classification task involves predicting one class label (eg, either cat or dog) per image. Ideal if there is only one main object in the image.
- Segmentation requires to predict the class label for every pixel in the image.

Segmentation: Problem Setup

- For classification task, we have training data of the format: $(\mathbf{X}, y)$, where $\mathbf{X} \in \mathbb{R}^{3 \times H \times W}$, with H and W being the height and width of the image; and y is the class label (one per input).
- In segmentation, we need supervision for every pixel of the input image. Therefore, the training data is of the format: $(\mathbf{X}, \mathbf{Y})$, where $\mathbf{Y} \in \mathbb{R}^{H \times W}$ is called the **segmentation map**.



- **Goal:** To predict an output segmentation map whose spatial dimensions is the same as the input image, ie, $H \times W$.

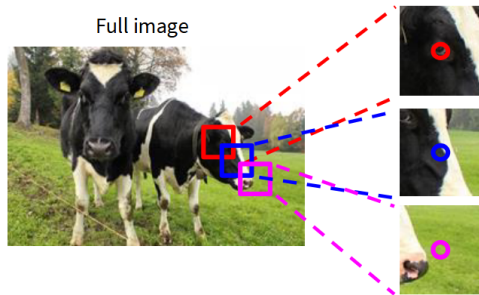
Segmentation: A Sliding Window Approach

Full image



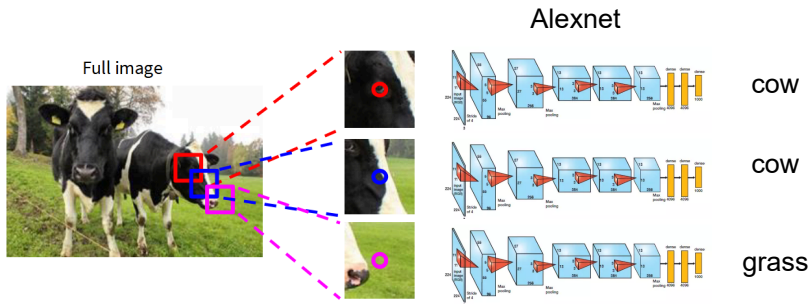
- A naive approach is to learn features from every pixel of the input.
- The issue is that a pixel does not contain enough information (or does not have enough context).

Segmentation: A Sliding Window Approach



- The idea is to extract a small window (or **patch**) centered around a given pixel (shown in colours).
- The small window provides contextual information to the model.
- Then we can slide the window across the whole image (that is why **sliding window**), and get a prediction for the **center pixel** of each window.

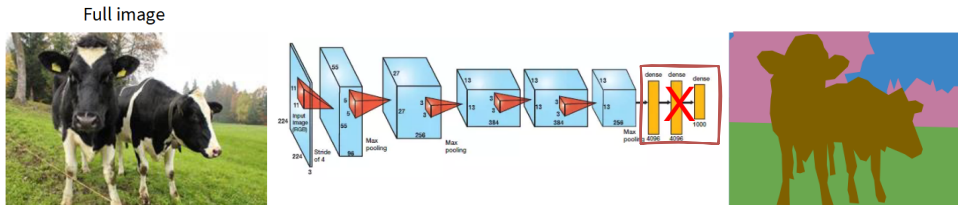
Segmentation: A Sliding Window Approach



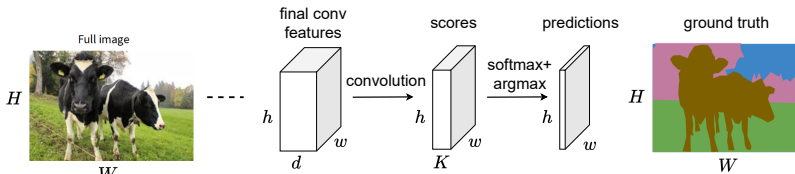
- A CNN (eg, Alexnet), takes as input an image patch. Just as in image classification, the CNN predicts **one class label** per patch, which is the label corresponding to the center pixel. This process is then repeated for every pixel location.
- The main drawback is **inefficiency**:
 - ▶ We need to run many forward passes, one per patch
 - ▶ Can not reuse shared features between overlapping patches.

Segmentation: Fully Convolutional Network

- The idea is to take as input the entire image and predict the output segmentation map by using the features.
- Modification needed: we replace the fully connected part at the end of the CNN with a convolutional layer.

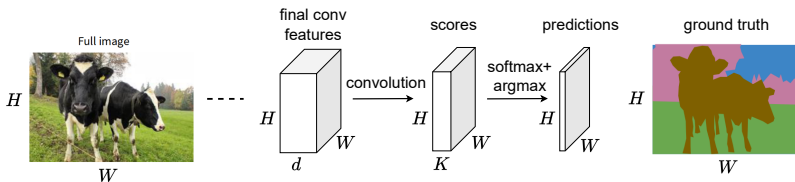


Segmentation: Fully Convolutional Network



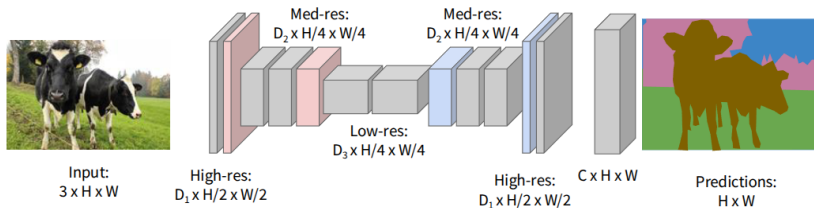
- We replace the fully connected layer with a convolutional layer. We apply convolution on the final conv features (shape: $[d \times h \times w]$) to obtain **scores** (shape: $[K \times h \times w]$), where K is the number of classes.
- After this we apply softmax + argmax to get the final predictions.
- **Downside:** due to the pooling operation in the CNN, the spatial dimensions of the feature maps are reduced, ie, $h < H$ and $w < W$.
- We can't supervise the training of such a network due to the mismatch of prediction and ground truth segmentation map.

Segmentation: Fully Convolutional Network



- While we could solve the issue of resolution by removing the pooling layers and keeping the spatial resolution intact, ie, all the feature maps will maintain its resolution of $H \times W$, we do not do that.
- Why? Because keeping the original means more higher computational complexity, and processing redundant information.

Segmentation: Fully Convolutional Network



- The network is designed to have:
 - ▶ a **downsampling** part (just as a regular CNN for image classification)
 - ▶ a **upsampling** part
- The downsampling part of the network produces lower resolution features: $D_3 \times H/4 \times W/4$ compared to the input image.
- The upsampling part of the network "upsamples" the lower resolution features back to the resolution of the image (or the ground truth): $\times H \times W$.
- **Question:** How can we model the upsampling operations?

Fully Convolutional Network: Upsampling

- Recap, in pooling (max-pooling or min-pooling) the goal is to go from a higher resolution feature map to a lower resolution feature map.
- To do the reverse, we do [upsampling](#) or "unpooling", which transforms the feature maps from a lower resolution to a higher resolution.
- There are several techniques for that:
 - ▶ Nearest neighbour
 - ▶ Bed of nails
 - ▶ Max unpooling
 - ▶ Transposed convolution

Fully Convolutional Network: Upsampling

- **Nearest neighbour** unpooling copies the value of a cell to its neighbouring locations, which is determined by a fixed kernel size (eg, 2×2).
- **"Bed of nails"** copies the value of a cell to the leftmost corner within the kernel, and sets all other elements to 0.
- In this example, we go from an input resolution of 2×2 to an output of resolution 4×4 .

Nearest Neighbor

1	2
3	4

Input: 2×2



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4×4

"Bed of Nails"

1	2
3	4

Input: 2×2

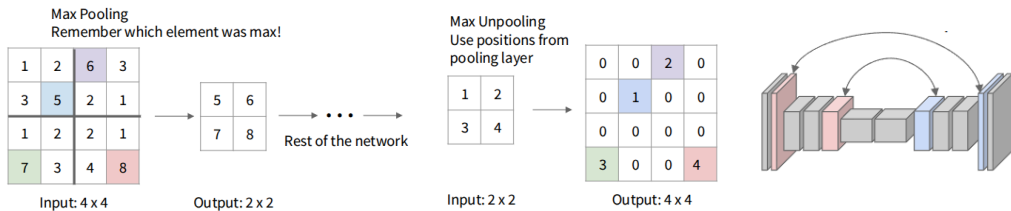


1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4×4

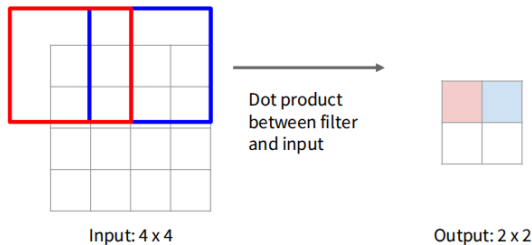
Fully Convolutional Network: Upsampling

- In **max unpooling** we assume that we have a mirrored downsampling and upsampling part of the network. Every pooling layer will have a corresponding unpooling layer.
- During pooling we additionally store the location of the max element (assuming max-pooling).
- During unpooling in the corresponding layer, we apply a sort of "bed of nails", but copy the value to the location of the maximum element.



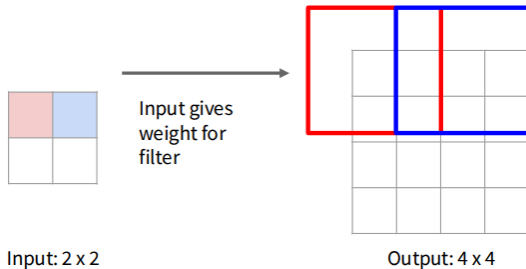
Fully Convolutional Network: Upsampling

- Recap, in regular convolution we move the kernel across the input (with or without padding) with a given stride, and compute the dot product between the kernel weights and the input.
- In the example below, we have used a 3×3 convolution with stride 2 and padding 1.
- Due to a stride of 2, we have now an output map which is half the size of the input.

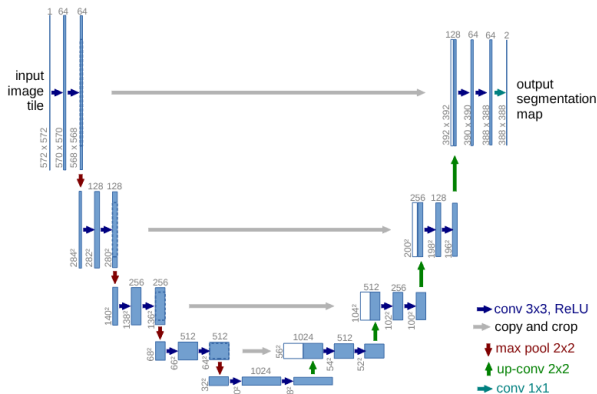


Fully Convolutional Network: Upsampling

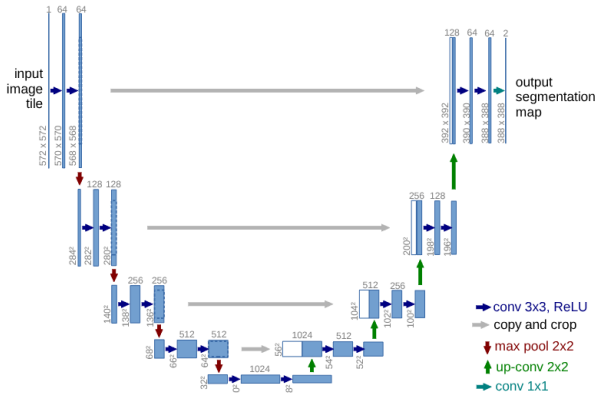
- To get the opposite effect, we use **transposed convolution**.
- In transposed convolution, the input gives weight for the filter. Since, our goal is to increase the resolution of the output, we use a stride of 2. We also use a padding of 1. The weights of the kernel are learned via backpropagation.
- In the overlapping regions, we sum the values.



U-Net

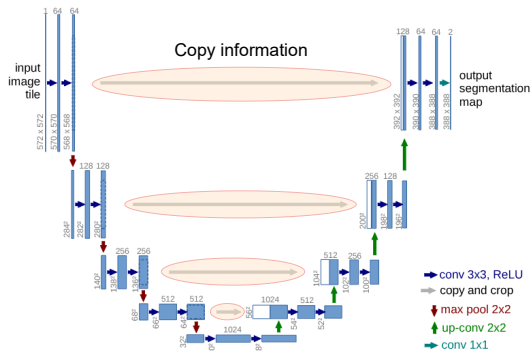


- U-Net is a fully convolutional architecture for image segmentation, originally designed for medical images, has become a popular architecture in many computer vision tasks, including segmentation, and image generation.



- A problem with the downsampling part of a fully convolutional network is that it loses spatial information due to pooling operations.
- Once these information are wiped out in the bottleneck layer, upsampling can not recover such information.

U-Net



- U-Net copies over the features of the corresponding layers of the downsampling part to the upsampling part of the network.
- This information is then concatenated along the channel dimensions (a wider feature maps) in the upsampling part.

References and Links

- For additional reading materials refer to the following links
 - ▶ [“The Illustrated Transformer”](#)².
 - ▶ [“Dive into Deep Learning”](#)³, Chapters: 11, 14.10, 14.11
- Some slides have been adapted from E. Adeli.

²<https://jalammr.github.io/illustrated-transformer/>

³<https://d2l.ai/>